

Documento de Casos de Teste — API de Tarefas

Módulo: src/tarefas/app.py
Arquivo de testes: tests/test_api.py
Framework: pytest

Endpoints testados

Rota	Método	Descrição
/auth/login	POST	Autenticação de usuário e geração de token JWT
/tarefas	POST	Criação de nova tarefa (Pydantic BaseModel)
/tarefas	GET	Listagem geral de tarefas cadastradas
/tarefas/{tarefa_id}	GET	Busca de uma tarefa específica por ID
/tarefas/{tarefa_id}	DELETE	Exclusão de tarefa (Exige autenticação JWT)

Casos de Teste

CT-01 — Login com credenciais válidas

ID	CT-01
Rota testada	POST /auth/login
Objetivo	Garantir a emissão do token ao fornecer credenciais válidas

Cenário/Entradas	username= 'aluno', password= 'senha123' (via form-data)
Resultado esperado	Status 200 e token JWT no corpo da resposta
Técnica	Inspeção de chaves no dicionário retornado
Tipo	Caminho feliz

```
def test_ct01_login_valido():
    resposta = client.post('/auth/login', data={'username': 'aluno', 'password': 'senha123'})
    assert resposta.status_code == 200
    dados = resposta.json()
    assert 'access_token' in dados
    assert dados['access_token'] != ""
    assert dados['token_type'] == 'bearer'
```

CT-02 — Login sem o campo username

ID	CT-02
Rota testada	POST /auth/login
Objetivo	Bloquear autenticação incompleta
Cenário/Entradas	Apenas password= 'senha123'
Resultado esperado	Status HTTP 422 Unprocessable Entity
Técnica	assert status_code == 422
Tipo	Validação de erro (Caminho alternativo)

```
def test_ct02_login_sem_username():
    resposta = client.post('/auth/login', data={'password': 'senha123'})
    assert resposta.status_code == 422
```

CT-03 — Login sem o campo password

ID	CT-03
Rota testada	POST /auth/login
Objetivo	Bloquear autenticação incompleta
Cenário/Entradas	Apenas username='aluno'
Resultado esperado	Status HTTP 422 Unprocessable Entity
Técnica	assert status_code == 422
Tipo	Validação de erro (Caminho alternativo)

```
def test_ct03_login_sem_password():
    resposta = client.post('/auth/login', data={'username': 'aluno'})
    assert resposta.status_code == 422
```

CT-04 — Criar tarefa com dados válidos

ID	CT-04
Rota testada	POST /tarefas
Objetivo	Garantir criação com sucesso
Cenário/Entradas	JSON com titulo e descricao preenchidos

Resultado esperado	Status 201, ID numérico e status "pendente"
Técnica	Verificação de tipos e igualdade de valores
Tipo	Caminho feliz

```
def test_ct04_criar_tarefa_com_dados_validos():
    payload = {"titulo": "Estudar pytest", "descricao": "Ler a doc"}
    resposta = client.post('/tarefas', json=payload)
    assert resposta.status_code == 201
    dados = resposta.json()
    assert isinstance(dados['id'], int)
    assert dados['titulo'] == payload['titulo']
    assert dados['status'] == 'pendente'
```

CT-05 — Criar tarefa sem descrição

ID	CT-05
Rota testada	POST /tarefas
Objetivo	Testar campo opcional do modelo Pydantic
Cenário/Entradas	JSON apenas com titulo
Resultado esperado	Status 201 e descrição retorna como null
Técnica	assert is None
Tipo	Comportamento opcional

```
def test_ct05_criar_tarefa_sem_descricao():
    payload = {"titulo": "Tarefa sem descricao"}
    resposta = client.post('/tarefas', json=payload)
    assert resposta.status_code == 201
    assert resposta.json()['descricao'] is None
```

CT-06 — Criar tarefa com título vazio

ID	CT-06
Rota testada	POST /tarefas
Objetivo	Garantir restrição de tamanho mínimo de string
Cenário/Entradas	{"titulo": ""}
Resultado esperado	Status 422 (Restrição do Field)
Técnica	Igualdade de status HTTP
Tipo	Limite inferior de fronteira

```
def test_ct06_criar_tarefa_com_titulo_vazio():
    resposta = client.post('/tarefas', json={"titulo": ""})
    assert resposta.status_code == 422
```

CT-07 — Criar tarefa sem o campo título

ID	CT-07
Rota testada	POST /tarefas
Objetivo	Garantir exigência de campo obrigatório

Cenário/Entradas	JSON sem a chave "titulo"
Resultado esperado	Status HTTP 422
Técnica	Igualdade de status HTTP
Tipo	Validação estrutural

```
def test_ct07_criar_tarefa_sem_campo_titulo():
    resposta = client.post('/tarefas', json={"descricao": "sem titulo"})
    assert resposta.status_code == 422
```

CT-08 — Criar tarefa com título acima do limite

ID	CT-08
Rota testada	POST /tarefas
Objetivo	Garantir restrição máxima de tamanho de string
Cenário/Entradas	String contendo 201 caracteres
Resultado esperado	Status HTTP 422
Técnica	Multiplicação de strings no Python "A" * 201
Tipo	Limite superior de fronteira

```
def test_ct08_criar_tarefa_com_titulo_acima_do_limite():
    resposta = client.post('/tarefas', json={"titulo": "A" * 201})
    assert resposta.status_code == 422
```

CT-09 — Status inicial da tarefa

ID	CT-09
Rota testada	POST /tarefas
Objetivo	Validar injeção obrigatória do status inicial
Cenário/Entradas	Criação padrão
Resultado esperado	Status na API retorna forçosamente "pendente"
Técnica	<code>assert json()['status'] == 'pendente'</code>
Tipo	Regra de negócio

```
def test_ct09_status_inicial_da_tarefa_criada():
    resposta = client.post('/tarefas', json={"titulo": "Validar Status"})
    assert resposta.status_code == 201
    assert resposta.json()['status'] == 'pendente'
```

CT-10 — Listar tarefas com repositório vazio

ID	CT-10
Rota testada	GET /tarefas
Objetivo	Validar resposta correta quando não há dados
Cenário/Entradas	Repositório em memória reiniciado
Resultado esperado	Status 200 e corpo como lista vazia []
Técnica	Igualdade estrutural de listas
Tipo	Estado inicial limpo

```
def test_ct10_listar_tarefas_com_repositorio_vazio():
    resposta = client.get('/tarefas')
    assert resposta.status_code == 200
    assert resposta.json() == []
```

CT-11 — Listar tarefas após criação

ID	CT-11
Rota testada	GET /tarefas
Objetivo	Validar listagem de objetos existentes
Cenário/Entradas	Criar um registro e em seguida consultar a listagem
Resultado esperado	Status 200 e o item recém-criado está presente na lista
Técnica	Manipulação de estado seguida de asserção de comprimento (len())
Tipo	Integração sequencial

```
def test_ct11_listar_tarefas_apos_criacao():
    client.post('/tarefas', json={"titulo": "Tarefa Listável"})
    resposta = client.get('/tarefas')
    assert resposta.status_code == 200
    assert len(resposta.json()) == 1
```

CT-12 — Buscar tarefa existente

ID	CT-12
Rota testada	GET /tarefas/{id}

Objetivo	Resgatar dados corretos de um ID específico
Cenário/Entradas	ID extraído da criação
Resultado esperado	Status 200 e mesmos dados de inserção
Técnica	Extração de variável dinâmica e rota formatada (f-string)
Tipo	Busca padrão

```
def test_ct12_buscar_tarefa_existente():
    criacao = client.post('/tarefas', json={"titulo": "Tarefa Especifica"})
    tarefa_id = criacao.json()['id']
    resposta = client.get(f'/tarefas/{tarefa_id}')
    assert resposta.status_code == 200
    assert resposta.json()['id'] == tarefa_id
```

CT-13 — Buscar tarefa inexistente

ID	CT-13
Rota testada	GET /tarefas/{id}
Objetivo	Garantir tratativa de ausência de dados
Cenário/Entradas	ID irreal 99999
Resultado esperado	Status 404 Not Found
Técnica	Igualdade de HTTP e checagem da mensagem de detalhe
Tipo	Verificação de erro esperado

```
def test_ct13_buscar_tarefa_com_id_inexistente():
    resposta = client.get('/tarefas/99999')
    assert resposta.status_code == 404
    assert resposta.json()['detail'] == 'Tarefa não encontrada'
```

CT-14 — Buscar tarefa com ID não numérico

ID	CT-14
Rota testada	GET /tarefas/{id}
Objetivo	Bloquear tipos de parâmetros errados na URL
Cenário/Entradas	Path parameter igual a "abc"
Resultado esperado	Status 422 Unprocessable Entity (Bloqueio do FastAPI)
Técnica	<code>assert status_code == 422</code>
Tipo	Validação de tipagem estática do endpoint

```
def test_ct14_buscar_tarefa_com_id_nao_numerico():
    resposta = client.get('/tarefas/abc')
    assert resposta.status_code == 422
```

CT-15 — Deletar tarefa sem token de autenticação

ID	CT-15
Rota testada	DELETE /tarefas/{id}
Objetivo	Bloquear exclusão anônima

Cenário/Entradas	Sem header de autorização
Resultado esperado	Status 401 Unauthorized
Técnica	Requisição padrão ao endpoint protegido
Tipo	Segurança e bloqueio de porta

```
def test_ct15_deletar_tarefa_sem_token():  
    resposta = client.delete('/tarefas/1')  
    assert resposta.status_code == 401
```

CT-16 — Deletar tarefa com token inválido

ID	CT-16
Rota testada	DELETE /tarefas/{id}
Objetivo	Bloquear forjamento de credenciais JWT
Cenário/Entradas	Header contendo assinatura falsa
Resultado esperado	Status 401 Unauthorized
Técnica	Uso de dicionário em headers= no TestClient
Tipo	Segurança

```
def test_ct16_deletar_tarefa_com_token_invalido():  
    headers = {"Authorization": "Bearer token-invalido"}  
    resposta = client.delete('/tarefas/1', headers=headers)  
    assert resposta.status_code == 401
```

CT-17 — Deletar tarefa existente com token válido

ID	CT-17
Rota testada	DELETE /tarefas/{id}
Objetivo	Fluxo de sucesso em rota protegida
Cenário/Entradas	Login, criação de tarefa, geração de token, requisição de deleção.
Resultado esperado	Status 204 No Content e texto de resposta em branco
Técnica	Fluxo completo end-to-end com autenticação
Tipo	Caminho feliz (Autenticado)

```
def test_ct17_deletar_tarefa_existente_com_token_valido():
    login_resp = client.post('/auth/login', data={'username': 'user', 'password': '123'})
    token = login_resp.json()['access_token']

    tarefa_id = client.post('/tarefas', json={"titulo": "Del"}).json()['id']

    headers = {"Authorization": f"Bearer {token}"}
    resposta = client.delete(f'/tarefas/{tarefa_id}', headers=headers)
    assert resposta.status_code == 204
```

CT-18 — Deletar tarefa inexistente com token válido

ID	CT-18
Rota testada	DELETE /tarefas/{id}
Objetivo	Validar verificação de existência do ID
Cenário/Entradas	Token real, ID irreal (99999)

Resultado esperado	Status 404 Not Found
Técnica	Fluxo de autenticação seguido de rota inexistente
Tipo	Exceção operacional

```
def test_ct18_deletar_tarefa_inexistente_com_token_valido():
    login_resp = client.post('/auth/login', data={'username': 'user', 'password': '123'})
    token = login_resp.json()['access_token']

    headers = {"Authorization": f"Bearer {token}"}
    resposta = client.delete('/tarefas/99999', headers=headers)
    assert resposta.status_code == 404
```

CT-19 — Tarefa deletada não é encontrada depois

ID	CT-19
Rota testada	Integração DELETE + GET
Objetivo	Garantir persistência da exclusão na memória
Cenário/Entradas	Apagar e consultar o mesmo ID
Resultado esperado	Status 404 Not Found na busca
Técnica	Teste de integração e fluxo cíclico
Tipo	Consistência de dados

```
def test_ct19_tarefa_deletada_nao_pode_ser_encontrada_depois():
    login_resp = client.post('/auth/login', data={'username': 'user', 'password': '123'})
    token = login_resp.json()['access_token']

    tarefa_id = client.post('/tarefas', json={"titulo": "Efêmera"}).json()['id']
    headers = {"Authorization": f"Bearer {token}"}
    client.delete(f'/tarefas/{tarefa_id}', headers=headers)

    busca = client.get(f'/tarefas/{tarefa_id}')
    assert busca.status_code == 404
```

Resumo das técnicas utilizadas no Pytest com FastAPI

Técnica (TestClient)	Quando usar
<code>client.post(url, json=...)</code>	Simular envio de um body cru JSON (como na criação de tarefas).
<code>client.post(url, data=...)</code>	Simular envio de um formulário x-www-form-urlencoded (Login do OAuth2).
<code>client.delete(url, headers=...)</code>	Injetar tokens Bearer manualmente para simular usuários validados.
<code>assert resposta.status_code == X</code>	Validação obrigatória de comportamento HTTP.
<code>@pytest.fixture(autouse=True)</code>	Executar um setup/teardown de ambiente global antes de todo teste.

Como executar os testes

A partir da raiz do projeto (onde encontra-se a pasta `tests` e o ambiente virtual ativado):

```
# Rodar todos os testes de forma resumida
pytest

# Rodar todos os testes exibindo os identificadores e a porcentagem de sucesso
pytest -v

# Rodar um teste em específico da suíte
pytest -v tests/test_api.py::test_ct01_login_valido
```

Resultado esperado ao rodar pytest -v

```
tests/test_api.py::test_ct01_login_valido PASSED [ 5%]
tests/test_api.py::test_ct02_login_sem_username PASSED [ 10%]
tests/test_api.py::test_ct03_login_sem_password PASSED [ 15%]
tests/test_api.py::test_ct04_criar_tarefa_com_dados_validos PASSED [ 21%]
tests/test_api.py::test_ct05_criar_tarefa_sem_descricao PASSED [ 26%]
tests/test_api.py::test_ct06_criar_tarefa_com_titulo_vazio PASSED [ 31%]
tests/test_api.py::test_ct07_criar_tarefa_sem_campo_titulo PASSED [ 36%]
tests/test_api.py::test_ct08_criar_tarefa_com_titulo_acima_do_limite PASSED [ 42%]
tests/test_api.py::test_ct09_status_inicial_da_tarefa_criada PASSED [ 47%]
tests/test_api.py::test_ct10_listar_tarefas_com_repositorio_vazio PASSED [ 52%]
tests/test_api.py::test_ct11_listar_tarefas_apos_criacao PASSED [ 57%]
tests/test_api.py::test_ct12_buscar_tarefa_existente PASSED [ 63%]
tests/test_api.py::test_ct13_buscar_tarefa_com_id_inexistente PASSED [ 68%]
tests/test_api.py::test_ct14_buscar_tarefa_com_id_nao_numerico PASSED [ 73%]
tests/test_api.py::test_ct15_deletar_tarefa_sem_token PASSED [ 78%]
tests/test_api.py::test_ct16_deletar_tarefa_com_token_invalido PASSED [ 84%]
tests/test_api.py::test_ct17_deletar_tarefa_existente_com_token_valido PASSED [ 89%]
tests/test_api.py::test_ct18_deletar_tarefa_inexistente_com_token_valido PASSED [ 94%]
tests/test_api.py::test_ct19_tarefa_deletada_nao_pode_ser_encontrada_depois PASSED [100%]
```

```
===== 19 passed in 0.52s =====
```